

Fortgeschrittene Informatik

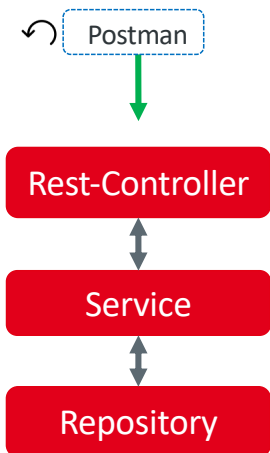
Prof. Dr. Andreas Schilling

www.ravensburg.dhbw.de

4. Testing

Motivation

- Unsere Webanwendung verfügt inzwischen über eine **REST-Schnittstelle**, die wir bereits mit Postman **manuell getestet** haben.
- Das Ziel von automatisierten Softwaretests ist ein **wiederholbares Ergebnis** unter **spezifizierten Voraussetzungen** zu erzielen.
- **Automatisierte Softwaretests** sind das **Rückgrat** der agilen Softwareentwicklung, da ihr Ergebnis den aktuellen Projektstand bestimmt.



Die Durchführung von Softwaretests stellt die korrekte Funktionalität einer Anwendung sicher. Softwaretests sind ein integraler Bestandteil der Softwareentwicklung, da sie sicherstellen, dass der aktuelle Arbeitsstand der Anwendung lauffähig ist und das gewünschte Verhalten erfüllt.



Grundlegende
Konventionen zur
Umsetzung von Tests.



Umsetzungs-
möglichkeiten von
Test-Doubles.



Vorgehen zum Test
von Controller-
methoden

Testing

Best-Practise

Benennung

- Testklassen beginnen mit der zu testenden Klasse und enden mit **„Test“**.
- Testmethoden beginnen mit **„should“** und dem zu testenden Verhalten.
- Testmethoden werden sowohl für das **erfolgreiche** und das **erfolglose** Verhalten verfasst.



Testumfang

- Ein Testfall umfasst **ein konkretes Verhalten**, nicht mehr & nicht weniger.
- Klasse stets ohne Seiteneffekte testen, außer bei Integration Tests.

5

Beim Verfassen von Softwaretests ist es wertvoll Best-Practice Empfehlungen für die Benennung und den Testumfang zu berücksichtigen um so die Verständlichkeit und die Übersichtlichkeit auch bei umfangreichen Tests zu gewährleisten. Es sollte darauf geachtet werden, dass die Testklasse mit dem Klassennamen der zu testenden Ressource beginnt und mit „Test“ endet. Beispiele für Testklassen sind:

- MovieControllerImplTest
- AwardControllerImplTest

Die Testmethoden innerhalb einer Testklasse sollten einen selbsterklärenden Namen tragen, der über das zu testende Verhalten Auskunft gibt und mit dem Präfix „should“ beginnt. Beispiele für die Benennung von Testmethoden sind:

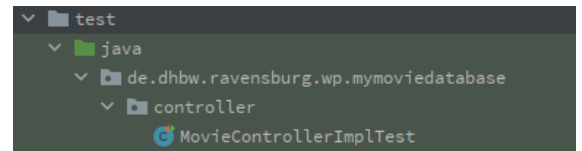
- ShouldNotLoginBecauseWrongPassword
- ShouldNotLoginBecauseEmptyPassword
- ShouldSuccessfullyLogin

Hinsichtlich dem Testumfang ist es empfehlenswert, isoliert das entsprechende Verhalten zu testen und kein weiteres darüber hinausgehendes Verhalten.

Testing

Grundlagen

- Der Testordner einer Spring-Boot Anwendung befindet sich im Ordner **src/test**.
- Innerhalb des Testordners sollte die **gleiche Ordnerstruktur (Package-Struktur)** wie für die **Spring-Boot Anwendung** verwendet werden.
- Bei Verwendung der gleichen Package-Struktur, können die zu testenden **Beans via Dependency Injection** in die Testklasse geladen werden.
- Für Softwaretests kann neben der **Constructor-Injection** auch die **Field-Injection** gemäß Best-Practice genutzt werden.



Gemäß der Best-Practice Konvention sollte der Test Ordner des Java-Projektes („src/test“) gleich wie der Ordner mit dem Quellcode („src“) strukturiert sein. Dies hat den Grund, dass es bei Verwendung der gleichen Ordnerstruktur möglich ist, die Dependency Injection von Spring zur Instanziierung der zu testenden Beans einzusetzen. Hierfür kann die Field-Injection über die Annotation `@Autowired` verwendet werden, da die Testklassen in der Regel alleinstehend sind.

Testing

JUnit

- **JUnit** ist ein **Testframework**, mit Hilfe dessen Testklassen verfasst und ausgeführt werden können.
- Hierbei wird jede Testmethode mit der Annotation **@Test** ausgezeichnet.
- Mit **Lifecycle Methoden** (@BeforeAll, @BeforeEach, @AfterAll, @AfterEach) können wiederverwendbare Testbedingungen für die Tests erstellt werden.

```
@Test
public void shouldSucessfullyReturnAllMovies()
```

Ausschnitt der Klasse MovieControllerImplTest

```
@BeforeEach
public void setUp(){

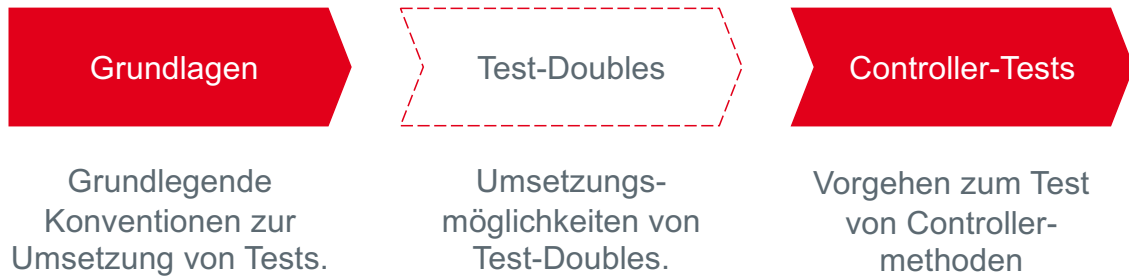
    AwardDTO oscar_1 = new AwardDTO( id: 1L, academy: "Oscar",
    AwardDTO oscar_2 = new AwardDTO( id: 2L, academy: "Oscar",
    AwardDTO oscar_3 = new AwardDTO( id: 3L, academy: "Oscar",

    movie_1 = new MovieDTO( id: 1L,
                           title: "Interstellar",
```

Ausschnitt der Klasse MovieControllerImplTest

Zum Verfassen der Testklassen verwenden wir das Testframework JUnit, das es ermöglicht Testfälle einfach und komfortabel zu verfassen. Hierzu wird über den Testmethoden die Annotation @Test platziert.

Beim Verfassen der Testmethoden ist darauf zu achten, dass sich wiederholender Code bezüglich des Aufbaus der zu testenden Objekte, in Hilfsmethoden ausgelagert wird. Hierzu können insb. die JUnit Lifecycle Methoden verwendet werden. So können die JUnit Annotationen @BeforeEach bzw. @BeforeAll genutzt werden um die Instanziierung und den Aufbau eines Objektes beim Ausführen der gesamten Testklasse (@BeforeAll) oder vor jeder Testmethode (@BeforeEach) vorzunehmen.



Testing

Definition

1. Unit-Test

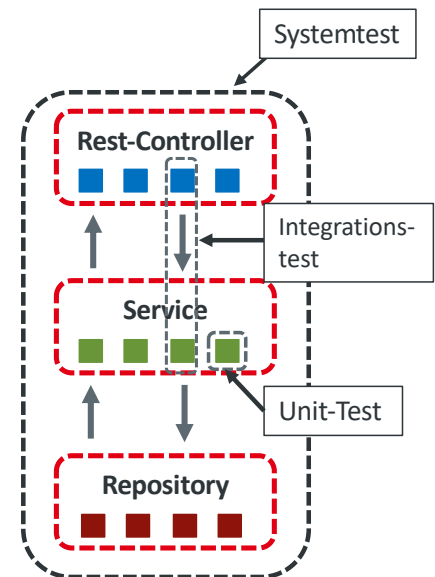
- **Ziel:** Überprüfung der korrekten Funktionsweise eines abgeschlossenen Softwarebausteins.
- Rückgrat der Softwarequalität und Sicherstellung der Geschäftslogik.

2. Integrationstests

- **Ziel:** Überprüfung des korrekten Zusammenspiels einzelner Komponenten.
- Prüfung der schichtenübergreifenden Interaktion.

3. Systemtest

- **Ziel:** Überprüfung des gesamten Systems inkl. nicht funktionaler Anforderungen.
- Testsystem soweit wie möglich an Produktivsystem anpassen.



Für die Umsetzung von Softwaretests werden drei grundlegende Formen von Tests unterschieden, die sich hinsichtlich ihrer Zielsetzung und ihres Testumfangs unterscheiden.

Die erste und elementarste Form der Softwaretests sind Unit-Tests. Diese Tests dienen dazu die korrekte Funktionsweise eines abgeschlossenen Softwarebausteins (bspw. einer Klassenmethode) sicherzustellen. Unit-Tests sind ein grundlegender Bestandteil der Softwareentwicklung, da sie den erzielten Projektfortschritt sicherstellen.

Die zweite Form von Tests sind Integrationstests. Diese Tests stellen sicher, dass das Zusammenspiel von Softwarebausteinen schichtenübergreifend funktioniert. Konkret wird hier sichergestellt, dass mehrere Softwarebausteine erfolgreich miteinander interagieren um die gewünschte Funktionalität umzusetzen.

Abschließend stellt der Systemtest das Zusammenspiel aller schichteninternen und schichtenübergreifenden Interaktionen sicher und testet Abläufe im gesamten System. Hierunter fallen sowohl funktionale als auch nicht-funktionale Anforderungen. Wichtig hierbei ist, dass das Testsystem so weit wie möglich dem Produktivsystem entsprechen sollte, damit die Testergebnisse repräsentativ sind.

Wir werden uns im Folgenden auf die Umsetzung der Unit-Tests fokussieren.

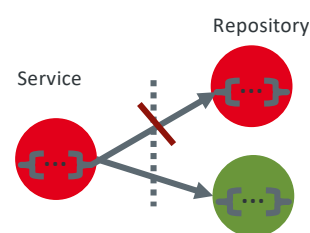
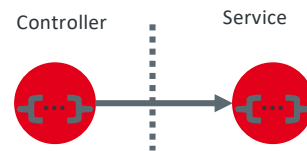
Testing

Test Double

- Eine zentrale Herausforderung für die Umsetzung von **Unit-Tests** sind Methoden, die **Abhängigkeiten zu anderen Klassen** haben, bspw.:
 - Interaktion von Klassen der gleichen Schicht
 - Schichtübergreifende Interaktionen

Wie lässt sich die Implementierung einer konkreten Methode testen, **ohne die aufgerufene Klasse einzubeziehen**?

- **Test Double** ist der Oberbegriff für den Austausch der aufgerufenen Klasse durch eine alternative Implementierung.



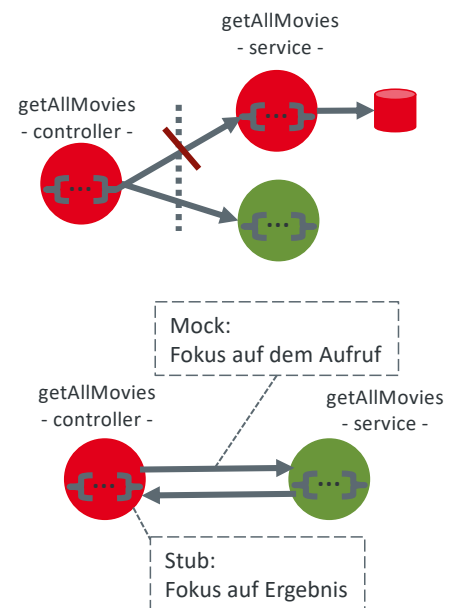
Eine große Herausforderung bei der Umsetzung der Unit-Tests betrifft die Vermeidung von Abhängigkeiten, da die zu testenden Methoden in der Regel nicht frei von Abhängigkeiten zu anderen Klassen sind. Im Falle von Spring hält ein Controller eine Referenz auf den Service und der Service wiederum eine Referenz auf das Repository. Doch wie lässt sich sicherstellen, dass das zu testende Verhalten isoliert und ohne die Auswirkungen der aufgerufenen Klasse getestet wird?

Eine Möglichkeit mit diesen Abhängigkeiten umzugehen ist die Nutzung eines Test-Double. Hierunter verbirgt sich die Idee, dass die tatsächliche Implementierung der aufgerufenen Klasse mit einer alternativen Implementierung ausgetauscht wird, sodass das Testverhalten isoliert ausgewertet werden kann. Es lassen sich drei unterschiedliche Formen von Test Doubles unterscheiden.

Testing

Arten von Test Doubles

- **Fake**
 - Eigenständig verfasste Alternativimplementierung.
 - Reduzierter Komplexitätsumfang ggü. Original.
 - Aufwändige manuelle Umsetzung und Wartung.
- **Mock**
 - Automatische Generierung
 - Gibt Standardwerte zurück
 - Evaluation des Verhaltens
- **Stub**
 - Semi-automatische Generierung
 - Angepasstes Antwortverhalten
 - Evaluation des Zustands



11

Die ursprüngliche Umsetzungsform für ein Test-Double ist der sogenannte Fake. Hierunter versteht man eine alternative Implementierung für die aufgerufene Klasse mit deutlich reduzierter Komplexität. Die Nutzung eines Fake eröffnet weitreichende Möglichkeiten zur Anpassung des Laufzeitverhaltens, jedoch ist für die Erstellung eines Fakes ein großer manueller Aufwand notwendig und die Implementierung muss fortwährend gepflegt werden.

Eine weitere Möglichkeit ein Test-Double umzusetzen ist die Verwendung eines Mocks. Ein Mock kann automatisch generiert werden. Bei dieser Form des Test-Double geht es nicht darum was für eine Rückgabe zurückkommt, sondern dass das korrekte Aufrufverhalten stattgefunden hat. Das heißt der Evaluationsfokus liegt hier auf der Überprüfung des Methodenaufrufs und nicht im zurückgegebenen Ergebnis.

Zuletzt kann ein Test-Double durch einen Stub umgesetzt werden. Der Stub kann halb-automatisch generiert werden, das heißt es kann eine Implementierung basierend auf dem genutzten Interface abgeleitet werden und dieses wird manuell um das konkrete Rückgabeverhalten angepasst. Beim Stub liegt der Evaluationsfokus auf der korrekten Weiterverwendung mit der Rückgabe des Methodenaufrufs.

Das folgende Beispiel aus der Finanzbranche stellt die Unterschiede zwischen Stub und Mock prägnant heraus. Nehmen wir an, wir entwickeln eine Java-Anwendung für das Onlinebanking und müssen ein Test-Double für den Bankserver erstellen. Bei Verwendung eines Stubs, würden wir das Antwortverhalten eines Kontos nachbilden (bspw. Stand 100,00€ -> Transaktion -> Stand 80,00€). Bei Verwendung eines Mocks würden wir dagegen „nur“ prüfen ob der Transaktionsbefehl korrekt an den Bankserver übermittelt wurde, der konkrete Kontostand und dessen Veränderung würden hier nicht

überprüft.

Testing

Test Doubles in Spring

▪ Vorbereitung

- Durch die Verwendung von **@MockBean** wird automatisch ein Mock für die Klasse erstellt.

```
@MockBean
MovieService movieService;
```

Ausschnitt der Klasse MovieControllerImplTest

▪ Mock

- Verwendung der **,verify’**-Syntax zur Validierung eines Aufrufs, sowie dessen Anzahl:
verify(mock, [times(n)]).methodName(Matcher)

```
verify(movieService).getAllMovies();
```

Ausschnitt der Klasse MovieControllerImplTest

▪ Stub

- **,when’ -> ,then’** Syntax bei Methoden mit Rückgabe, bei void Methoden **do -> when** Syntax.
- Auch Exceptions können geworfen werden.

```
@Test
public void shouldSuccessfullyReturnAllMovies() throws Exception{
    when(movieService.getAllMovies()).thenReturn(movieList);
}
```

Ausschnitt der Klasse MovieControllerImplTest

Schauen wir uns zum Abschluss an wie Stubs und Mocks bei Spring Boot verwendet werden. Die Voraussetzung hierfür ist die Verwendung der Annotation **@MockBean** für die zu „mockenden“ Komponenten. Hiermit wird Spring angewiesen nicht die „echte“ Implementierung der referenzierten Bean zu verwenden, sondern automatisch eine Mock-Implementierung für das Interface abzuleiten. Diese Mock-Implementierung gibt nur die Standardwerte der Parameter zurück (d.h. null bei Objekten, 0 bei int).

Als nächstes wird im Rahmen einer Testmethode der generierte Mock verwendet um die Interaktionsbeziehungen zwischen den Objekten zu validieren. Mit der Methode **verify** wird überprüft, ob der spezifizierte Methodenaufwurf tatsächlich stattgefunden hat.

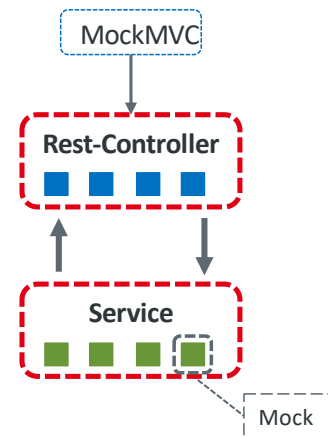
Des Weiteren kann der generierte Mock in ein Stub überführt werden indem das gewünschte Antwortverhalten der automatisch generierten Mock-Implementierung manuell in Form der **when-then** Syntax hinzugefügt wird. Hierbei wird im **when**-Teil die Aufruf-Spezifikation definiert und im **then**-Teil die genaue Rückgabe des Stubs.



Controller-Tests

Überblick

- In der Spring-Boot Anwendung greifen die **REST-Controller** auf die **Funktionalität der Services** zu.
- Doch die **Repositories und damit auch die Services** haben bei jedem Testdurchlauf einen **unterschiedlichen Zustand**.
- Um reproduzierbare Testbedingungen herzustellen muss die Umsetzung der **Services gemockt** werden.
- Des weiteren wird Postman durch **MockMVC ersetzt**, um die Rückgabe des Rest-Controllers **automatisiert auszuwerten**.



Für die Umsetzung von Unit-Tests bei den REST-Controllern gilt es verschiedene Rahmenbedingungen zu berücksichtigen. So sind die Rest-Controller nicht alleinstehend, sondern von den dazugehörigen Services abhängig. Wie im letzten Kapitel detailliert, birgt diese Abhängigkeit jedoch Probleme in der Testausführung, da abhängig von der vorhergehenden Testausführung unterschiedliche Zustände in der Datenbank und dadurch in den Repositories und damit in den Services vorliegen können.

Es ist daher unumgänglich sicherzustellen, dass die gleichen Rahmenbedingungen für alle Tests vorliegen. Dies lässt sich erreichen indem die Umsetzung der Services in den Controllern gemockt wird.

Eine weitere Veränderung für die automatisierte Ausführung der Softwaretests ist der Austausch von Postman mit dem Werkzeug MockMVC. Dieses Werkzeug ermöglicht die Rückgaben des Rest-Controllers automatisiert auszuwerten.

Controller-Tests

MockMVC

- MockMVC ist ein Test Framework, das es erlaubt die **REST-Annotationen** des Controllers automatisiert zu testen.
- Wird eine Testklasse mit **@WebMvcTest** ausgezeichnet, wird ein Application-Context erzeugt, der nur die referenzierte Controller Bean beinhaltet.
- Services können mit der Annotation **@MockBean** eingebunden werden.
- Das **MockMvc Objekt** kann man sich wie ein Kommandozeilenwerkzeug für REST-Aufrufe vorstellen.
- Der **ObjectMapper** dient dazu bei den POST und PUT Operationen ein Objekt zu übertragen.

```
@WebMvcTest(controllers = MovieControllerImpl.class)
public class MovieControllerImplTest {

    @MockBean
    private MovieService movieService;

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    //Test Entities
    private List<MovieDTO> movieList;
    private MovieDTO movie_1;
    private MovieDTO movie_2;

    private final String controllerPath = "/api/v1/movies";

    @BeforeEach
    public void setUp(){

        AwardDTO oscar_1 = new AwardDTO( id: 1L, academy: "Oscar",
        AwardDTO oscar_2 = new AwardDTO( id: 2L, academy: "Oscar",
        AwardDTO oscar_3 = new AwardDTO( id: 3L, academy: "Oscar",
```

Ausschnitt der Klasse MovieControllerImplTest

15

Bei MockMVC handelt es sich um ein Test-Framework das es ermöglicht die REST-spezifischen Annotationen des RestControllers auszuwerten. Um das Framework zu verwenden wird über die Testklasse die Annotation **@WebMvcTest** angebracht und der zu testende Controller spezifiziert. MockMVC erstellt daraufhin einen Application-Context der nur aus der spezifizierten Controller Bean besteht. Alle weiteren Beans werden über die Annotation **@MockBean** als Mock eingebunden. Des Weiteren wird das MockMvc Object und der ObjectMapper via Field-Injection in die Testklasse eingebunden. Das MockMvc Objekt dient im weiteren Verlauf dazu, ähnlich wie Postman, REST-Anfragen an den REST-Controller zu stellen. Der ObjectMapper wird verwendet um in den POST- und PUT-Anfragen Objekte im Inhaltsteil der HTTP-Anfrage zu übertragen.

Controller-Tests

MockMVC - GetAbfrage

- Zu Beginn des Tests wird der zu **mockende Service-Aufruf** inkl. der Rückgaben detailliert.
- Ähnlich wie bei Postman wird bei MockMvc der **Anfragetyp** und **Anfragepfad** übergeben
- Die Rückgabe wird anschließend mit Hilfe von **jsonPath** evaluiert:



- Abschließend wird **sichergestellt**, dass der Mock tatsächlich **aufgerufen wurde**.

```

@Test
public void shouldSuccessfullyReturnMovie() throws Exception{
    when(movieService.getMovieById(1L)).thenReturn(movie_1);
    this.mockMvc.perform(get(urlTemplate: controllerPath+"/1"))
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$.*", hasSize(9)))
        .andExpect(jsonPath( expression: "$.id", is(movie_1.getId().intValue())))
        .andExpect(jsonPath( expression: "$.title", is(movie_1.getTitle())))
        .andExpect(jsonPath( expression: "$.duration", is(movie_1.getDuration())))
        .andExpect(jsonPath( expression: "$.description", is(movie_1.getDescription())))
        .andExpect(jsonPath( expression: "$.trailerUrl", is(movie_1.getTrailerUrl())))
        .andExpect(jsonPath( expression: "$.coverImgUrl", is(movie_1.getCoverImgUrl())))
        .andExpect(jsonPath( expression: "$.imdbRating", is(movie_1.getImdbRating())))
        .andExpect(jsonPath( expression: "$.castIds", containsInAnyOrder(movie_1.getCastIds())))
        .andExpect(jsonPath( expression: "$.awards[0].id", is(movie_1.getAwards().get(0).getId().intValue())))
        .andExpect(jsonPath( expression: "$.awards[0].academy", is(movie_1.getAwards().get(0).getAcademy())))
        .andExpect(jsonPath( expression: "$.awards[0].category", is(movie_1.getAwards().get(0).getCategory())))
        .andExpect(jsonPath( expression: "$.awards[0].celebrationYear", is(movie_1.getAwards().get(0).getCelebrationYear().intValue())))
        .andDo(print());
    verify(movieService).getMovieById(1L);
}

```

Ausschnitt der Klasse MovieControllerImplTest

Ein Testfall wird bei MockMVC mit der Annotation `@Test` ausgezeichnet. Hierbei wird zuerst das gewünschte Verhalten des Mocks spezifiziert (bspw. was soll bei welchem Input zurückgegeben werden). Als nächstes wird der tatsächliche REST-Aufruf der HTTP-Schnittstelle ausgeführt, hierbei wird der vollständige Kontext-Pfad samt der verwendeten Parameter spezifiziert. Daran anschließend wird die Rückgabe des Aufrufs evaluiert. Bei der Evaluation der Rückgabe werden die zu überprüfenden Werte via JSON-Path Notation aufgerufen. Der Pfad „`$.description`“ verwendet beispielsweise das erste Objekt der Rückgabe und von diesem das Attribut „`description`“.

Zuletzt findet die Überprüfung statt, dass der Mock für die Ausführung des REST-Aufrufs tatsächlich aufgerufen wurde.

Controller-Tests

MockMVC – Post-Aufruf

- Zu Beginn des Tests wird der **zu mockende Service-Aufruf** inkl. Rückgabe spezifiziert.
- Bei **POST- und PUT-Aufrufen** wird der Anfrageinhalt mithilfe des objectMappers und der Methode `writeValueAsString` übergeben.
- Die Rückgabe wird anschließend mit Hilfe von `jsonPath` evaluiert.
- Abschließend wird **sichergestellt**, dass der Mock tatsächlich **aufgerufen wurde**.

```
@Test
public void shouldSuccessfullyAddMovie() throws Exception{
    when(movieService.addMovie(any())).thenReturn(movie_2);
    this.mockMvc.perform(post(controllerPath)
        .contentType(MediaType.APPLICATION_JSON)
        .content(this.objectMapper.writeValueAsString(movie_1))
        .andExpect(status().isCreated())
        .andExpect(jsonPath(expression: "$.id", is(movie_2.getId)))
        .andExpect(jsonPath(expression: "$.title", is(movie_2.getTitle)))
        .andExpect(jsonPath(expression: "$.duration", is(movie_2.getDuration())))
        .andExpect(jsonPath(expression: "$.description", is(movie_2.getDescription())))
        .andExpect(jsonPath(expression: "$.trailerUrl", is(movie_2.getTrailerUrl())))
        .andExpect(jsonPath(expression: "$.coverImgUrl", is(movie_2.getCoverImgUrl())))
        .andExpect(jsonPath(expression: "$.imdbRating", is(movie_2.getImdbRating())))
    )
    verify(movieService).addMovie(any());
}
```

Ausschnitt der Klasse `MovieControllerImplTest`

Eine HTTP-Post Anfrage wird in ähnlicher Weise spezifiziert. Auch hier wird zunächst der Testfall mit der Annotation `@Test` gekennzeichnet. Daran anschließend wird in der Methode das Verhalten des Mocks spezifiziert, also welche Antwort auf welches Eingabeverhalten zurückgegeben werden soll. Als nächstes wird der HTTP-POST Aufruf spezifiziert. Im Gegensatz zum GET-Aufruf wird hierbei zusätzlich der Content-Type und der konkrete Inhalt spezifiziert. Für die Übergabe des Inhalts kommt der ObjectMapper mit der Methode `writeValueAsString` zum Einsatz, die ein Java-Objekt in ein String überführt. Im Anschluss an die Spezifikation der Anfrage, wird der zurückgegebene HTTP-Statuscode und der Inhalt überprüft.

Den Abschluss der Testmethode bildet die `verify`-Methode, die sicherstellt, dass der spezifizierte Mock bei der Bearbeitung der REST-Abfrage ausgeführt wurde.

Controller-Tests

MockMvc – Fehlverhalten Testen

- Zum Testen der Fehlerbehandlung, wird im **Mock** die Bedingung und die konkrete **Exception** spezifiziert.
- Anschließend daran erfolgt der MockMvc-Aufruf (hier bspw. mit Anfrageinhalt)
- Der **Statuscode** wird gemäß der im Controller **spezifizierten Rückgabe** überprüft.
- Abschließend wird **sichergestellt**, dass der Mock tatsächlich **aufgerufen wurde**.

```
@Test
public void shouldFailGetMovie() throws Exception{
    when(movieService.getMovieById(anyLong())).
        thenThrow(new NoSuchElementException("No movie found"));
    this.mockMvc.perform(get(urlTemplate: controllerPath+"/20"))
        .andExpect(status().isNotFound());
    verify(movieService).getMovieById(20L);
}
```

Ausschnitt der Klasse MovieControllerImplTest

Schauen wir uns zum Abschluss den Test eines fehlerhaften Verhaltens an. Nach dem Anbringen der Test-Annotation wird der Mock spezifiziert. Hier wird statt eines konkreten Ergebnisses eine Exception zurückgegeben. Der Typ der Exception hängt vom zu testenden Fehlverhalten ab. Im oben abgebildeten Fall, soll beispielsweise die Verletzung einer Validierungsvorschrift nachgestellt werden. Als Nächstes wird im oberen Beispiel der MockMvc-Aufruf inklusive contentType und Anfrage-Inhalt ausgeführt und es wird überprüft ob der im Controller spezifizierte ResponseCode im Fehlerfall zurückgegeben wird. Den Abschluss bildet die Überprüfung, dass die spezifizierte Methode des Service-Mocks vom Controller im Rahmen der Abfrage auch verwendet wurde.

Controller-Tests

Hands-On

- Konfiguration der Testklasse
- Umsetzung des Testfalls
shouldSuccessfullyReturnAllMovies
- Umsetzung des Testfalls
shouldFailGetMovie
- Erläuterungen zur Spezifikation der **Mocks**

```
@WebMvcTest(controllers = MovieControllerImpl.class)
public class MovieControllerImplTest {

    @MockBean
    private MovieService movieService;

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    //Test Entities
    private List<MovieDTO> movieList;
    private MovieDTO movie_1;
    private MovieDTO movie_2;

    private final String controllerPath = "/api/v1/movies";

    @BeforeEach
    public void setUp(){

        AwardDTO oscar_1 = new AwardDTO( id: 1L, academy: "Oscar",
        AwardDTO oscar_2 = new AwardDTO( id: 2L, academy: "Oscar",
        AwardDTO oscar_3 = new AwardDTO( id: 3L, academy: "Oscar",
```

Ausschnitt der Klasse MovieControllerImplTest

Aufgabe 6

Controller Test & DTO

Erweitern Sie die Testklasse MovieControllerImplTest mit den folgenden Testmethoden:

- a) **shouldSuccessfullyUpdateMovie**
- b) **shouldFailUpdateMovieBecauseUnknownId**
- c) **shouldFailRemoveMovieBecauseUnknownId**
- d) **shouldFailUpdateMovieBecauseConstraintViolation**
- e) **shouldFailAddNewMovieDueToConstraintViolation**
- f) **shouldSuccessfullyRemoveMovieById**
- g) **shouldFailRemoveMovieBecauseUnknownId**